



نام درس: شبکه های کامپیوتری
نویسنده: محمد امانلو - ۸۱۰۱۰۰۰۸۴
مهراد لیویان - ۸۱۰۱۰۱۵۰۱



تمرین
کامپیوتری دوم
فاز دوم

۱ مقدمه و هدف پروژه

در این فاز، هدف ما طراحی و پیاده سازی یک جریان UDP-مبنا است که در حضور جریان (های) TCP روی یک لینک گلوگاه مشترک، رفتار TCP-Friendly داشته باشد. همان طور که در صورت پروژه آمده است، UDP ذاتاً مکانیزم کنترل ازدحام ندارد و اگر یک جریان UDP با نرخ ثابت در کنار TCP قرار گیرد، معمولاً سهم بزرگی از پهنای باند را تصاحب می کند و باعث starvation جریان TCP می شود. بنابراین، در این پروژه ما:

□ سه سناریوی TCP-only، TCP+UDP بدون کنترل و TCP+UDP با کنترل نرخ TCP-Friendly را شبیه سازی می کنیم؛

□ معیارهای Throughput، average delay، packet loss و Jain's fairness index را استخراج و مقایسه می کنیم؛

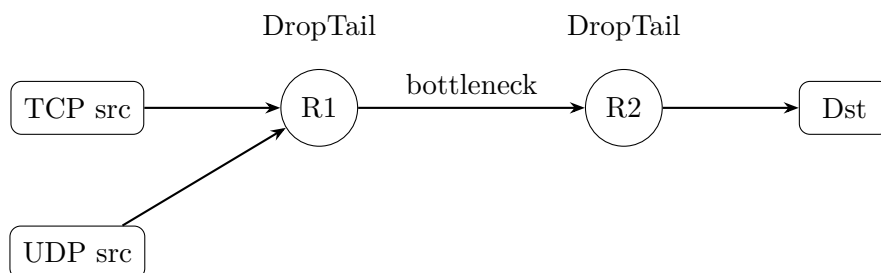
□ نشان می دهیم که الگوریتم ساده ی افزایش/کاهش پیشنهادی (شبیه TFRC-like) چگونه باعث TCP-friendly شدن UDP می شود.

۲ توپولوژی شبکه و تنظیمات شبیه سازی

توپولوژی پروژه از نوع shared bottleneck است: جریان های TCP و UDP از یک لینک مشترک R1-R2 عبور می کنند و روی این گلوگاه رقابت دارند.

۱.۲ شکل توپولوژی

در شکل ۱ توپولوژی مورد استفاده را (به صورت شماتیک) نشان داده ایم.



شکل ۱: توپولوژی shared bottleneck مورد استفاده در این فاز.

۲.۲ فرض ها و پارامترهای لینک

طبق گزارش ما، پارامترهای لینک ها به صورت زیر تنظیم شده اند (واحد پهنای باند Mb/s و واحد تأخیر ms است):

لینک	پهنای باند	تأخیر
R1 ↔ TCP src	10 Mb	10 ms
R1 ↔ UDP src	10 Mb	10 ms
R2 ↔ R1 (گلوگاه)	2 Mb	20 ms
Dst ↔ R2	10 Mb	10 ms

جدول ۱: تنظیمات لینک ها در شبیه سازی.

همچنین ظرفیت بافر روی لینک گلوگاه R1-R2 برابر 20 (به صورت queue-limit) و سیاست صف DropTail در نظر گرفته شده است. برای جریان TCP نیز از TCP NewReno استفاده کرده ایم.

۳.۲ زمان شبیه سازی

در فایل ها، شروع جریان ها را از زمان 1.0s قرار داده ایم و پایان شبیه سازی حدود 50s است. این کار باعث می شود اثرات warm-up اولیه کمتر در میانگین ها اثر بگذارد.

۳ سناریوهای شبیه سازی

طبق صورت پروژه، سه سناریو را اجرا کرده ایم:

۱.۳ سناریو 1: TCP-only (خط مبنا)

در این حالت فقط یک جریان TCP از مبدأ به مقصد برقرار است. این سناریو به عنوان baseline استفاده می شود تا ببینیم در نبود رقابت، TCP چگونه از ظرفیت گلوگاه استفاده می کند.

۲.۳ سناریو 2: TCP + UDP بدون کنترل نرخ (UDP Raw)

در این حالت یک جریان TCP و یک جریان UDP همزمان فعال اند و UDP با نرخ ثابت (بدون کنترل ازدحام) ارسال می کند. انتظار می رود UDP سهم بیشتری از پهنای باند را بگیرد و شاخص انصاف کاهش یابد.

۳.۳ سناریو 3: TCP + UDP با کنترل نرخ (UDP Friendly)

در این حالت، همان توپولوژی سناریوی 2 را داریم اما نرخ UDP را با یک الگوریتم ساده TFRC-like تنظیم می کنیم تا UDP در برابر ازدحام (به ویژه packet loss) واکنش نشان دهد و تقسیم منابع منصفانه تر شود.

۴ پیاده سازی در NS-2

در گزارش ما، برای هر سناریو یک فایل TCL ساخته ایم:

tcp_only.tcl □

tcp_udp_raw.tcl □

tcp_udp_friendly.tcl □

۱.۴ ایجاد فایل های TCL

ما ابتدا پوشه ی فاز دوم را ایجاد کردیم و فایل های TCL را ساختیم (شکل های ۲ و ۳).

```
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2$ ls
phase1
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2$ mkdir phase2
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2$ ls
phase1 phase2
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2$ cd phase2
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ ls
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ nano tcp_only.tcl
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ |
```

شکل ۲: ایجاد پوشه ی phase2 و شروع ساخت فایل tcp_only.tcl.

```
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAS/2/phase2$ ls
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAS/2/phase2$ nano tcp_only.tcl
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAS/2/phase2$ nano tcp_udp_raw.tcl
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAS/2/phase2$ nano tcp_udp_friendly.tcl
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAS/2/phase2$ |
```

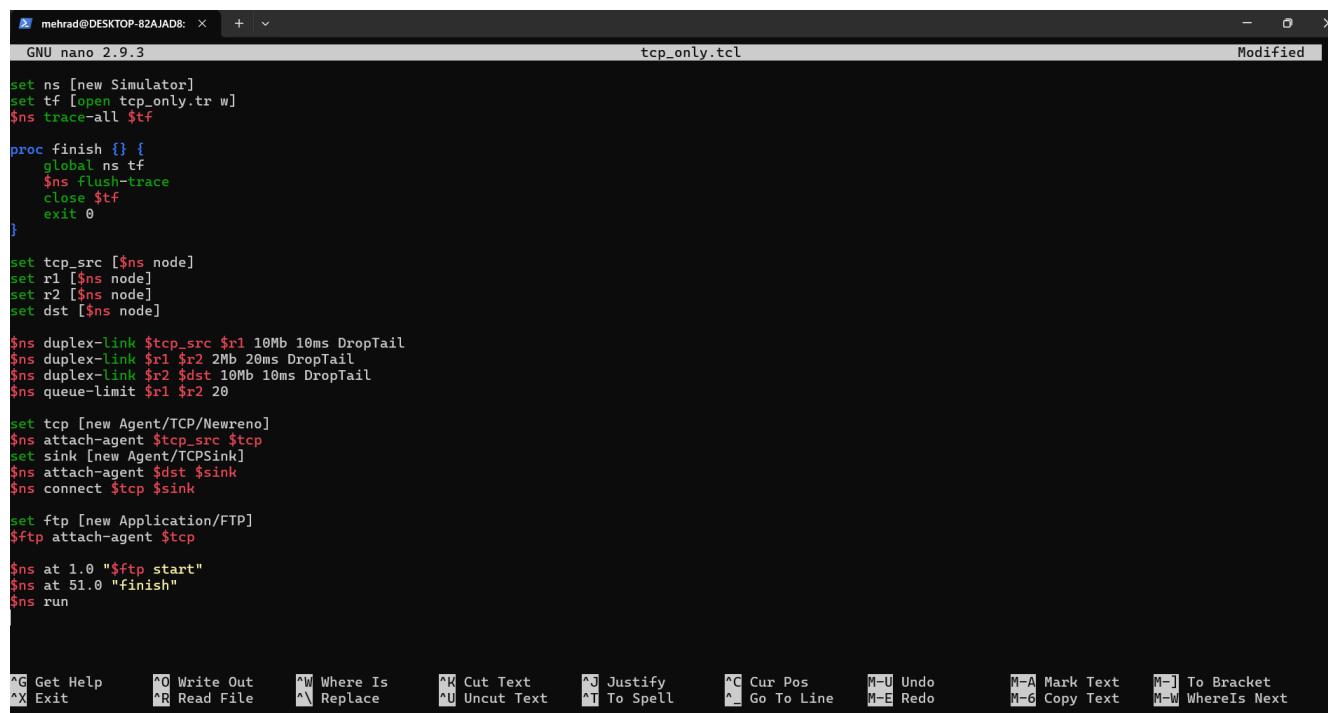
شکل ۳: ساخت سه فایل TCL مربوط به سناریوها.

```
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAS/2/phase2$ nano tcp_udp_raw.tcl
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAS/2/phase2$ nano tcp_udp_friendly.tcl
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAS/2/phase2$ nano analyzer.py
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAS/2/phase2$ |
```

شکل ۴: ویرایش فایل های tcp_udp_raw.tcl و tcp_udp_friendly.tcl و همچنین analyzer.py.

۲.۴ سناریو TCP-only: فایل tcp_only.tcl

در این سناریو، تنها یک مبدأ TCP، دو روتر R1 و R2 و یک مقصد داریم. لینک ها و صف مطابق جدول ۱ تنظیم شده اند و TCP از نوع NewReno است. برای تولید ترافیک TCP از کاربرد FTP استفاده کرده ایم. نمایی از فایل tcp_only.tcl در شکل ۵ آمده است.



```
GNU nano 2.9.3 tcp_only.tcl Modified

set ns [new Simulator]
set tf [open tcp_only.tr w]
$ns trace-all $tf

proc finish {} {
    global ns tf
    $ns flush-trace
    close $tf
    exit 0
}

set tcp_src [$ns node]
set r1 [$ns node]
set r2 [$ns node]
set dst [$ns node]

$ns duplex-link $tcp_src $r1 10Mb 10ms DropTail
$ns duplex-link $r1 $r2 2Mb 20ms DropTail
$ns duplex-link $r2 $dst 10Mb 10ms DropTail
$ns queue-limit $r1 $r2 20

set tcp [new Agent/TCP/Newreno]
$ns attach-agent $tcp_src $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $dst $sink
$ns connect $tcp $sink

set ftp [new Application/FTP]
$ftp attach-agent $tcp

$ns at 1.0 "$ftp start"
$ns at 51.0 "finish"
$ns run
```

شکل ۵: نمایی از پیاده سازی سناریوی TCP-only در فایل tcp_only.tcl.

۳.۴ سناریو UDP Raw: فایل tcp_udp_raw.tcl

در این سناریو، علاوه بر جریان TCP، یک جریان UDP نیز اضافه می‌کنیم. برای UDP از Agent/UDP و در مقصد از Agent/Null استفاده کرده‌ایم و در سطح کاربرد نیز یک CBR تعریف کرده‌ایم تا UDP با نرخ ثابت ارسال کند. در شکل ۶ بخشی از این فایل نشان داده شده است.

```

set ns [new Simulator]
set tf [open tcp_udp_raw.tr w]
$ns trace-all $tf

proc finish {} {
    global ns tf
    $ns flush-trace
    close $tf
    exit 0
}

set tcp_src [$ns node]
set udp_src [$ns node]
set r1 [$ns node]
set r2 [$ns node]
set dst [$ns node]

$ns duplex-link $tcp_src $r1 10Mb 10ms DropTail
$ns duplex-link $udp_src $r1 10Mb 10ms DropTail
$ns duplex-link $r1 $r2 2Mb 20ms DropTail
$ns duplex-link $r2 $dst 10Mb 10ms DropTail
$ns queue-limit $r1 $r2 20

set tcp [new Agent/TCP/Newreno]
$ns attach-agent $tcp_src $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $dst $sink
$ns connect $tcp $sink

set udp [new Agent/UDP]
$ns attach-agent $udp_src $udp
set null [new Agent/Null]
$ns attach-agent $dst $null
$ns connect $udp $null

set cbr [new Application/Traffic/CBR]

```

شکل ۶: نمایی از فایل tcp_udp_raw.tcl (سناریوی TCP+UDP بدون کنترل نرخ).

۴.۴ سناریو UDP Friendly: فایل tcp_udp_friendly.tcl

در این سناریو، توپولوژی دقیقاً با سناریوی UDP Raw یکسان است (طبق گزارش ما)، اما نرخ UDP به صورت پویا تنظیم می‌شود. منطق کنترل نرخ دقیقاً مطابق الگوریتم پیشنهادی صورت پروژه است:

□ در صورت مشاهده loss: Multiplicative Decrease $\alpha \leftarrow 0.85 \times \alpha$

□ در صورت گذشت یک RTT بدون loss: Additive Increase $rate \leftarrow rate + \alpha$ $\alpha = 0.01 \times rate$

در پیاده‌سازی ما، نرخ در یک متغیر rate نگهداری می‌شود و یک پرچم loss_flag مشخص می‌کند که آیا در بازه‌ی اخیر loss رخ داده است یا خیر. سپس روی یک دوره‌ی زمانی (هم‌اندازه با RTT یا یک بازه‌ی ثابت نزدیک به آن) تابع update_rate اجرا می‌شود و نرخ CBR را به‌روزرسانی می‌کند.

بخشی از کد کنترل نرخ در شکل ۷ نشان داده شده است.

```

GNU nano 2.9.3 tcp_udp_friendly.tcl Modified
set ns [new Simulator]
set tf [open tcp_udp_friendly.tr w]
$ns trace-all $tf

set rate 1.0
set loss_flag 0

proc update_rate {} {
    global rate loss_flag cbr
    if {$loss_flag == 1} {
        set rate [expr $rate * 0.85]
        set loss_flag 0
    } else {
        set rate [expr $rate + 0.01*$rate]
    }
    $cbr set rate_ "${rate}Mb"
}

proc finish {} {
    global ns tf
    $ns flush-trace
    close $tf
    exit 0
}

set tcp_src [$ns node]
set udp_src [$ns node]
set r1 [$ns node]
set r2 [$ns node]
set dst [$ns node]

$ns duplex-link $tcp_src $r1 10Mb 10ms DropTail
$ns duplex-link $udp_src $r1 10Mb 10ms DropTail
$ns duplex-link $r1 $r2 2Mb 20ms DropTail
$ns duplex-link $r2 $dst 10Mb 10ms DropTail
$ns queue-limit $r1 $r2 20

^G Get Help      ^O Write Out    ^W Where Is     ^K Cut Text     ^J Justify      ^C Cur Pos      M-U Undo        M-A Mark Text   M-J To Bracket
^X Exit          ^R Read File    ^_ Replace      ^U Uncut Text   ^T To Spell     ^_ Go To Line   M-E Redo        M-G Copy Text   M-W WhereIs Next

```

شکل ۷: بخشی از فایل tcp_udp_friendly.tcl و پیاده سازی کنترل نرخ TFRC-like.

تشخیص loss و RTT. طبق رویکرد پیشنهادی در صورت پروژ، ساده ترین راه برای تشخیص ازدحام، مانیتور کردن صف لینک گلوگاه است. ما loss را از طریق مشاهده ی رخ داده های drop (در صف DropTail لینک R1-R2) تشخیص می دهیم و با رخ دادن هر drop، مقدار loss_flag را 1 قرار می دهیم تا در اولین فراخوانی update_rate نرخ کاهش یابد.

برای زمان بندی به روزرسانی نرخ، می توان از RTT تخمینی استفاده کرد. در تنظیمات ما، تأخیر انتشار یک طرفه مسیر Src→Dst برابر است با:

$$10\text{ms} + 20\text{ms} + 10\text{ms} = 40\text{ms}$$

بنابراین RTT پایه (بدون در نظر گرفتن تأخیر صف) حدود 80ms است. در عمل، با اضافه شدن queueing delay، RTT موثر می تواند بزرگ تر شود؛ اما برای یک کنترل ساده ی TFRC-like، همین مقیاس زمانی برای update کافی است.

۵ اجرای خودکار سناریوها و تحلیل نتایج با Python

برای اینکه اجرای سناریوها و جمع‌بندی نتایج منظم باشد، یک اسکریپت Python با نام analyzer.py نوشتیم که:

۱. هر سه فایل TCL را پشت‌سرهم اجرا می‌کند؛

۲. این کار را ۵ بار تکرار می‌کند (در این پروژه ذاتاً randomness نداریم، اما اگر داشتیم میانگین‌گیری چند اجرا ضروری بود)؛

۳. خروجی هر اجرا را با نظم در پوشه‌ی results/ ذخیره می‌کند؛

۴. در انتها، میانگین معیارهای عملکرد را گزارش می‌دهد.

۱.۵ نمایی از analyzer.py

شکل ۸ بخشی از کد analyzer.py را نشان می‌دهد که در آن تعداد اجراها، زمان شبیه‌سازی و لیست سناریوها تعریف شده است.

```

import os
import shutil
import numpy as np

RUNS = 5
SIM_TIME = 50.0

SCENARIOS = {
    "tcp_only": {
        "tcl": "tcp_only.tcl",
        "trace": "tcp_only.tr"
    },
    "udp_raw": {
        "tcl": "tcp_udp_raw.tcl",
        "trace": "tcp_udp_raw.tr"
    },
    "udp_friendly": {
        "tcl": "tcp_udp_friendly.tcl",
        "trace": "tcp_udp_friendly.tr"
    }
}

BASE_RESULT_DIR = "results"

def parse_trace(filename):
    recv_bytes = {}
    send_times = {}
    delays = []

    is_tcp_only = "tcp_only" in filename

    if is_tcp_only:
        bottleneck_from = "1"
        bottleneck_to = "2"
    else:
        bottleneck_from = "2"

```

شکل ۸: بخشی از فایل analyzer.py: تعریف سناریوها و شروع تابع تحلیل.

۲.۵ ساختار خروجی ها

همان طور که در شکل ۹ مشاهده می شود، خروجی ها را به صورت `results/<scenario>/<run>` ذخیره کرده ایم تا مقایسه و بازتولید نتایج ساده باشد.

```
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ ls results/
tcp_only  udp_friendly  udp_raw
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ ls results/tcp_only/
1 2 3 4 5
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ ls results/tcp_only/1
tcp_only.tr
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ head results/tcp_only/1/tcp_only.tr
+ 1 0 1 tcp 40 ----- 0 0.0 3.0 0 0
- 1 0 1 tcp 40 ----- 0 0.0 3.0 0 0
r 1.010032 0 1 tcp 40 ----- 0 0.0 3.0 0 0
+ 1.010032 1 2 tcp 40 ----- 0 0.0 3.0 0 0
- 1.010032 1 2 tcp 40 ----- 0 0.0 3.0 0 0
r 1.030192 1 2 tcp 40 ----- 0 0.0 3.0 0 0
+ 1.030192 2 3 tcp 40 ----- 0 0.0 3.0 0 0
- 1.030192 2 3 tcp 40 ----- 0 0.0 3.0 0 0
r 1.040224 2 3 tcp 40 ----- 0 0.0 3.0 0 0
+ 1.040224 3 2 ack 40 ----- 0 3.0 0.0 0 1
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ |
```

شکل ۹: ساختار پوشه ی `results/` و نمونه ای از فایل `trace` تولیدشده.

۳.۵ اجرای اسکریپت و مشاهده خروجی

پس از نوشتن `analyzer.py`، آن را اجرا کرده ایم. شکل ۱۰ اجرای ۵ ران و محاسبه ی `final averages` را نشان می دهد.

```
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ nano analyzer.py
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ python3 analyzer.py

=== RUN 1 ===
→ Running tcp_only, run 1
→ Running udp_raw, run 1
→ Running udp_friendly, run 1

=== RUN 2 ===
→ Running tcp_only, run 2
→ Running udp_raw, run 2
→ Running udp_friendly, run 2

=== RUN 3 ===
→ Running tcp_only, run 3
→ Running udp_raw, run 3
→ Running udp_friendly, run 3

=== RUN 4 ===
→ Running tcp_only, run 4
→ Running udp_raw, run 4
→ Running udp_friendly, run 4

=== RUN 5 ===
→ Running tcp_only, run 5
→ Running udp_raw, run 5
→ Running udp_friendly, run 5

===== FINAL AVERAGES =====

TCP_ONLY
Avg Throughput : 1.921 Mbps
Avg Packet Loss: 10.00
Avg Delay      : 15.29 ms
Jain Fairness  : 1.000

-----
UDP_RAW
Avg Throughput : 1.998 Mbps
Avg Packet Loss: 249.00
Avg Delay      : 37.91 ms
```

شکل ۱۰: اجرای `analyzer.py` و اجرای تکرارشونده ی سناریوها.

در نهایت، خلاصه‌ی نتایج میانگین (برای هر سه سناریو) مطابق شکل ۱۱ به دست آمده است.

```
→ Running udp_friendly, run 4
=== RUN 5 ===
→ Running tcp_only, run 5
→ Running udp_raw, run 5
→ Running udp_friendly, run 5

===== FINAL AVERAGES =====
TCP_ONLY
Avg Throughput : 1.921 Mbps
Avg Packet Loss: 10.00
Avg Delay      : 15.29 ms
Jain Fairness  : 1.000
-----
UDP_RAW
Avg Throughput : 1.998 Mbps
Avg Packet Loss: 249.00
Avg Delay      : 37.91 ms
Jain Fairness  : 0.625
-----
UDP_FRIENDLY
Avg Throughput : 1.994 Mbps
Avg Packet Loss: 195.00
Avg Delay      : 33.14 ms
Jain Fairness  : 0.933
-----
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/2/phase2$ |
```

شکل ۱۱: خلاصه‌ی میانگین معیارها (Avg Throughput, Avg Packet Loss, Avg Delay و Jain Fairness).

۶ معیارهای ارزیابی و نحوه محاسبه

برای تحلیل خروجی‌ها، چهار معیار اصلی را محاسبه کرده‌ایم.

۱.۶ Throughput

Throughput را به صورت نسبت کل داده‌ی موفقِ تحویل گرفته‌شده در مقصد به طول بازه‌ی زمانی اندازه‌گیری تعریف می‌کنیم. اگر B_{recv} تعداد بایت‌های دریافت‌شده در مقصد و Δt طول بازه باشد:

$$\text{Throughput} = \frac{\Delta B_{recv}}{\Delta t}$$

۲.۶ Throughput بر حسب زمان (اختیاری)

علاوه بر میانگین کلی، برای مشاهده‌ی پویایی زمانی و پایداری جریان‌ها، می‌توان throughput را در پنجره‌های زمانی ثابت (مثلاً هر 0.5s) محاسبه کرد. در این روش، بازه‌ی زمانی شبیه‌سازی را به پنجره‌های $[t, t + \Delta]$ تقسیم می‌کنیم و برای هر پنجره، مقدار داده‌ی دریافت‌شده را شمرده و از رابطه‌ی زیر throughput پنجره را می‌گیریم:

$$\text{Throughput}(t) = \frac{\Delta B_{recv}(t, t + \Delta)}{\Delta}$$

این نمودارها کمک می‌کنند اثر رخدادهای ازدحام (مانند drop) و واکنش کنترل نرخ UDP Friendly (کاهش سریع و افزایش آهسته) را به صورت بصری ببینیم؛ دقیقاً همان چیزی که صورت پروژه از ما خواسته است.

۳.۶ Packet Loss

برای packet loss دو روش رایج وجود دارد: (۱) شمارش رخدادهای drop در لینک گلوگاه، یا (۲) اختلاف بین تعداد بسته‌های ارسال شده و دریافت شده. در تحلیل ما، از اطلاعات trace برای شمارش drop (و/یا اختلاف ارسال/دریافت) استفاده کرده‌ایم.

۴.۶ Average Delay

برای هر بسته، اگر زمان ارسال t_s و زمان دریافت t_r باشد، تأخیر آن بسته برابر $t_r - t_s$ است و میانگین تأخیر از میانگین‌گیری روی تمام بسته‌های دریافت شده به دست می‌آید:

$$\overline{D} = \frac{1}{N} \sum_{k=1}^N (t_r^{(k)} - t_s^{(k)})$$

۵.۶ Jain's Fairness Index

اگر $\{x_i\}_{i=1}^n$ throughput جریان‌ها باشد، شاخص انصاف Jain به صورت زیر تعریف می‌شود:

$$J = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2}$$

مقدار J در بازه $[0, 1]$ است و هرچه به 1 نزدیک‌تر باشد، تقسیم منابع منصفانه‌تر است.

۷ نتایج عددی و تحلیل

در این بخش، نتایج میانگین سه سناریو را (طبق خروجی analyzer.py) گزارش و تفسیر می‌کنیم.

۱.۷ جدول خلاصه نتایج

Jain Fairness	(ms) Avg Delay	Avg Packet Loss	(Mb/s) Avg Throughput	سناریو
۰.۰۰.۱	۲۹.۱۵	۰.۰.۱۰	۹۲۱.۱	TCP_ONLY
۶۲۵.۰	۹۱.۳۷	۰.۰.۲۴۹	۹۹۸.۱	UDP_RAW
۹۳۳.۰	۱۴.۳۳	۰.۰.۱۹۵	۹۹۴.۱	UDP_FRIENDLY

جدول ۲: خلاصه‌ی نتایج میانگین برای سه سناریو (مطابق خروجی شکل ۱۱).

۲.۷ تحلیل سناریو TCP-only

در سناریوی خط مبنا، تنها جریان TCP فعال است و بنابراین شاخص انصاف Jain طبیعی است که 1.0 شود. Throughput نزدیک به ظرفیت لینک گلوگاه 2Mb است (حدود 1.921 Mb/s) و مقدار loss و delay نیز پایین تر از حالت های رقابتی است. این سناریو نشان می دهد که TCP NewReno می تواند در غیاب رقیب، به شکل پایدار از گلوگاه استفاده کند.

۳.۷ تحلیل سناریو UDP Raw

در این سناریو، UDP با نرخ ثابت ارسال می کند و به سیگنال های ازدحام حساس نیست. در نتیجه:

□ Packet loss به شکل چشمگیری افزایش یافته است (حدود 249 در میانگین) که نشان می دهد صف DropTail گلوگاه بارها سرریز شده است.

□ Average delay نیز به دلیل تجمع صف و رخدادهای ازدحام افزایش یافته است.

□ شاخص انصاف Jain برابر 0.625 است که بیانگر تقسیم ناعادلانه تر پهنای باند بین دو جریان است.

این دقیقاً با انتظار پروژه سازگار است: UDP بدون کنترل می تواند جریان TCP را تحت فشار قرار دهد و تقسیم منابع را بدتر کند.

۴.۷ تحلیل سناریو UDP Friendly

در این سناریو، UDP به محض مشاهده ی loss نرخ را با ضریب 0.85 کاهش می دهد و در نبود loss به آهستگی افزایش می دهد. اثر این مکانیزم در نتایج کاملاً قابل مشاهده است:

□ شاخص انصاف از 0.625 به 0.933 افزایش یافته است؛ یعنی تقسیم منابع بین UDP و TCP به شکل معنی داری منصفانه تر شده است.

□ Packet loss از 249 به 195 کاهش یافته است؛ یعنی کنترل نرخ باعث شده ازدحام کمتر (یا کنترل شده تر) باشد.

□ Average delay نیز نسبت به حالت UDP Raw کاهش یافته است (از 37.91ms به 33.14ms).

□ در عین حال، Throughput کل همچنان نزدیک به ظرفیت گلوگاه باقی مانده است (حدود 1.994 Mb/s)؛ یعنی کنترل نرخ باعث افت شدید کارایی نشده و فقط تقسیم را منصفانه تر کرده است.

۵.۷ برآورد تقریبی سهم هر جریان از روی Jain (در صورت دو جریان)

در سناریوهای TCP+UDP ما دو جریان داریم ($n=2$). اگر throughput دو جریان را x_1 و x_2 بنامیم، شاخص Jain برابر است با:

$$J = \frac{(x_1 + x_2)^2}{2(x_1^2 + x_2^2)}$$

اگر علاوه بر J ، مجموع $x_1 + x_2$ را نیز داشته باشیم (که در خروجی ما به عنوان Avg Throughput گزارش شده است)، می توان نسبت دو جریان را تخمین زد. برای UDP Raw با $J \simeq 0,625$ نسبت دو جریان حدود $x_{\max}/x_{\min} \approx 7,87$ می شود و اگر $x_1 + x_2 \simeq 1,998 \text{ Mbps}$ باشد، به طور تقریبی:

$$x_{\max} \approx 1,773 \text{ Mbps}, \quad x_{\min} \approx 0,225 \text{ Mbps}.$$

برای UDP Friendly با $J \simeq 0,933$ نسبت دو جریان حدود $x_{\max}/x_{\min} \approx 1,73$ و با $x_1 + x_2 \simeq 1,994 \text{ Mbps}$ خواهیم داشت:

$$x_{\max} \approx 1,264 \text{ Mbps}, \quad x_{\min} \approx 0,730 \text{ Mbps}.$$

نکته: این بخش یک برآورد است و تنها در صورتی دقیق است که Avg Throughput واقعاً مجموع دو جریان باشد. با این حال، این تخمین به خوبی نشان می دهد چرا در حالت UDP Raw یک جریان عملاً غالب می شود، اما در حالت UDP Friendly سهم ها به هم نزدیک تر می شوند (که با انتظار پروژه نیز سازگار است).

۶.۷ نشان دادن TCP-Friendly شدن UDP با الگوریتم کاهش-افزایش

ایده ی اصلی در TCP friendliness این است که UDP هم مانند TCP به ازدحام واکنش نشان دهد:

□ واکنش سریع به ازدحام: با رخداد loss، نرخ به صورت ضربی کاهش می یابد (Multiplicative Decrease). این مشابه کاهش cwnd در TCP پس از تشخیص ازدحام است.

□ افزایش آهسته در نبود ازدحام: با گذشت یک بازه (در حد RTT) بدون loss، نرخ کمی زیاد می شود (Additive Increase). این مشابه فاز Congestion Avoidance در TCP است.

نتیجه ی این چرخه این است که UDP دیگر با نرخ ثابت به شبکه فشار نمی آورد و در صورت پر شدن صف/افت بسته، عقب نشینی می کند. بنابراین جریان TCP فرصت می یابد سهم معقولی از پهنای باند را نگه دارد و شاخص انصاف بهبود یابد (مطابق جدول ۲).

۸ جمع بندی

در این پروژه ما سه سناریوی اصلی را در NS-2 پیاده سازی و اجرا کردیم و نشان دادیم:

□ در حالت TCP-only، جریان TCP می تواند به صورت پایدار از گلوگاه استفاده کند؛

□ اضافه شدن UDP بدون کنترل، موجب افزایش loss و delay و کاهش fairness می شود؛

□ اعمال الگوریتم ساده ی TFRC-like روی UDP، بدون افت محسوس کارایی، شاخص انصاف را به شکل معنی داری افزایش می دهد و UDP را TCP-friendly می کند.

مراجع